

大模型时代下的存储系统挑战与技术发展

冯杨洋, 汪庆, 舒继武

清华大学计算机科学与技术系, 北京 100084

摘要

大语言模型（简称大模型）在文本和视觉处理等复杂任务中表现出色，受到工业界和学术界的广泛关注。大模型的训练与推理高度依赖于 GPU 算力，而 GPU 的显存容量有限且属于易失性存储介质，难以满足大模型在训练和推理过程中的存储需求。深入分析了大模型时代下存储系统面临的挑战：一是大模型的数据呈现高度碎片化特征，且大模型数据语义稀疏化显著，这降低了存储系统的利用率；二是大模型训练与推理对数据的读写带宽需求高，但异构存储介质间的数据传输通信开销大，这增加了利用异构存储介质扩展 GPU 显存的难度；三是大模型训练过程中的容错需求高，但直接使用以 CPU 为中心的容错技术会带来高昂的开销。针对上述挑战，从数据管理、存储扩容和数据容错 3 个方面总结了现有的解决方案。最后，展望了未来大模型时代存储系统的发展趋势。

关键词

大模型；存储系统；数据管理；存储扩容；数据容错

中图分类号: TP391

文献标志码: A

doi: 10.11959/j.issn.2096-0271.2025001

Challenges and technical development of storage systems in the era of large language models

FENG Yangyang, WANG Qing, SHU Jiwu

Department of Computer Science and Technology, Tsinghua University, Beijing 100084, China

Abstract

Large language models have shown exceptional performance in complex tasks, such as text and visual processing, drawing significant attention from industry and academia. Both the training and inference phases of large language models are heavily reliant on GPU computational power. However, the limited capacity and volatility of GPU memory pose significant challenges in meeting the storage demands during these phases. This paper conducts an in-depth analysis of the challenges faced by storage systems in the era of large language models: (1) the data of large language models exhibits a high degree of fragmentation, and sparse semantic representation further reduces storage system efficiency; (2) large language model training and inference demand high data read/write bandwidth, but the high communication overhead associated with data transfer across heterogeneous storage media complicates the expansion of GPU memory using these media; (3) the fault tolerance requirements during large language model training are stringent, yet directly applying CPU-centric fault tolerance techniques incurs prohibitive costs. To address these

challenges, existing solutions are summarized from three perspectives: data management, storage expansion, and fault tolerance. Finally, the future development trends of storage systems in the era of large language models are discussed.

Key words
large language model, storage system, data management, storage scalability, data fault tolerance

0 引言

OpenAI 推出的人工智能聊天机器人 ChatGPT 引发了工业界和学术界对大语言模型（即大模型）及相关应用的广泛关注。大模型凭借其卓越的理解与生成能力，已被广泛应用于金融、教育、艺术设计、游戏、医药和娱乐等多个领域。规模理论^[1]指出大模型优异的算法性能得益于模型参数量的增长。例如，Google 公司 2017 年推出的 Transformer 模型^[2]采用自

注意力机制提高了文本并行处理能力，该模型参数规模首次突破亿级。OpenAI 公司 2020 年发布了 GPT-3，其参数规模达到了千亿级别，在文本生成方面展现出极高的连贯性和多样性。2024 年，Meta 公司推出的 Llama 3.1 的参数规模较 GPT-3 提升了两倍，其不仅在文本处理上取得了更高的精度，还在多模态任务中展现出优异的算法性能。

随着大模型规模的不断增长，其在训练和推理过程中产生的数据量不断增加，对存储的需求也日益提升^[3]。但是，大模型时代下的存储系统以 GPU 为中心，这给存储系统带来了新的挑战。首先，GPU 的存算资源具有强耦合的特点。在大模型训练与推理过程中，模型数据（如模型参数、KV Cache 等）需存储在高读写带宽的 GPU 显存中，以匹配高吞吐的计算资源。然而，GPU 的存储资源增长与大模型存储需求增长不匹配，这使得 GPU 的计算资源受存储资源限制，导致其计算资源难以被充分利用。如图 1 所示，从 Transformer 到 GPT-3，模型训练计算量每翻一倍，模型参数量就会增长 20%；而 GPU 硬件的算力每翻一倍，存储容量仅增长 8%。其次，以 GPU 为中心的存储系统的故障概率高。大模型训练通常需要耗费海量的计算资源和大量时间，增加了训练过程中出现软硬件故障的风险。例如，Meta 团队在训练 OPT-175B 时出现了百余次故障^[4]，其中最长的连续无故障训练时间仅为 2.8 天。然而，由于 GPU 显存属于易失存

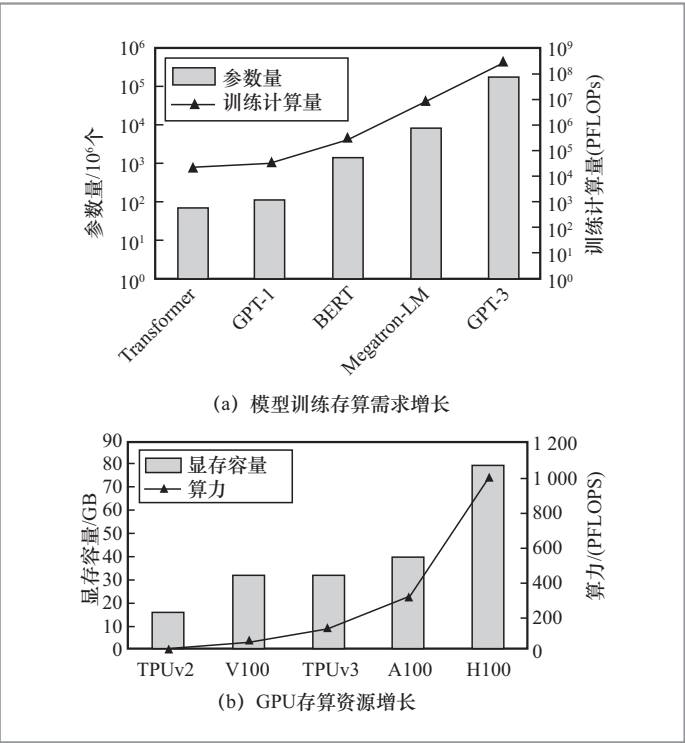


图 1 模型训练存算需求与 GPU 存算资源增长趋势

储介质，故障会造成训练数据丢失。

针对大模型时代下存储资源稀缺的问题，存储系统不仅需要精细化管理训练与推理过程中的数据^[5]，以充分利用有限的存储硬件资源，还需要考虑如何利用GPU服务器中各类异构存储资源，以扩充有限的GPU显存。同时，针对大模型时代下存储故障频发的问题，存储系统的设计不仅要增强数据的容错性，还要降低数据容错的开销，以减少对计算性能的影响。

本文将分别探讨大模型时代存储系统在数据管理、存储扩容和数据容错方面面临的挑战，并梳理现有研究工作针对这些挑战提出的解决方案；从绿色存储、云端协同存储以及系统与算法协同设计3个角度，对未来面向大模型的存储系统的发展趋势进行展望。

1 大模型时代下的数据管理

大模型训练与推理阶段均会产生大量的临时数据，这些数据展现出独有的特性。

本节将从大模型训练与推理阶段出发，分析数据特征及其对存储系统数据管理的挑战，并进一步探讨应对这些挑战的解决思路。

1.1 大模型数据特征与管理挑战

1.1.1 训练阶段

大模型训练会产生大量临时数据（如激活量与梯度），且临时数据会在产生后的较长时间后再次被使用。如图2所示，大模型训练主要包括前向传播、反向传播和参数更新3个阶段。首先，在前向传播阶段，训练数据依次通过模型的各层进行计算，并产生激活量。激活量不仅作为下一层的输入，还会被保存以供反向传播使用。其次，前向传播结束后，计算结果与训练数据的标签比对，生成损失（即误差）用于反向传播。在反向传播阶段，误差以与前向传播相反的方向，与各层参数以及前向传播中保存的该层激活量进行计算，并产生每一层参数对应的梯度。当某层的梯度计算完成后，其激活量可被释放，而梯

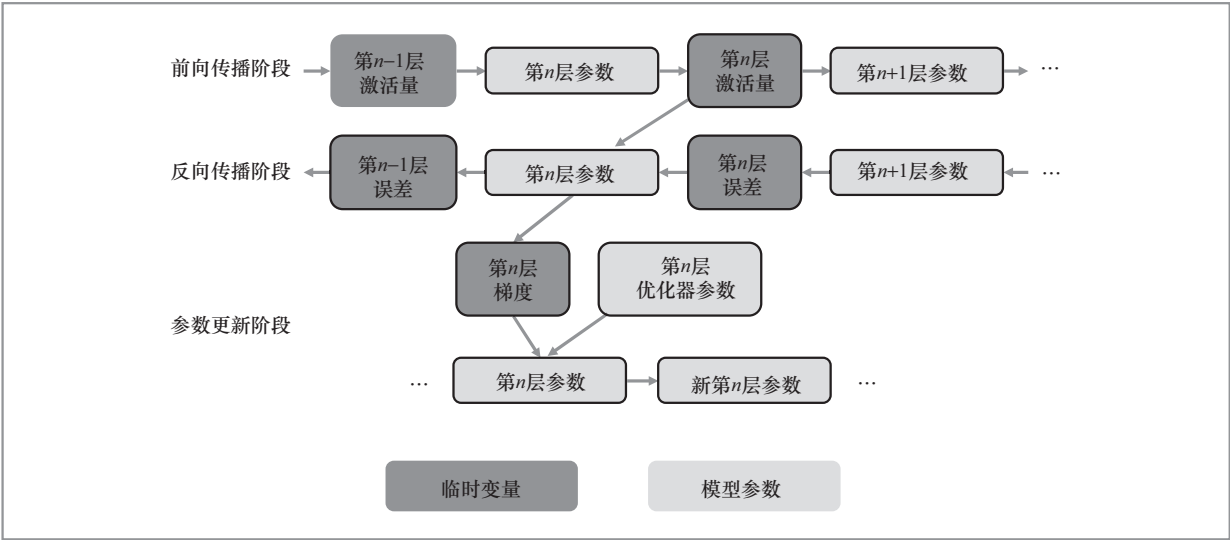


图2 大模型训练中各阶段的数据依赖

度数据需要被保留用于参数更新。最后，在参数更新阶段，优化器根据保存的梯度以及其自身的参数（如学习率、动量等）更新模型的参数。同时，优化器的内部参数也会根据当前的梯度信息进行更新。参数更新完成后，上一版本的模型参数、梯度数据以及优化器参数被释放。训练模型会使用更新后的模型参数与新的训练数据重复上述过程，逐步提升模型的性能。

大模型的训练模式导致临时数据使用率低，降低了GPU有限显存空间的利用率。例如，梯度数据需等待反向传播计算结束后才被用于参数更新；前向传播时每一层计算得到的激活量需要在反向传播计算后才能被释放。这些临时数据会在产生较长时间后再次被使用，导致GPU有限的存储资源长期被占用。

1.1.2 推理阶段

在大模型推理阶段，人们难以预测每个请求生成的临时数据量（即KV Cache）。大模型采用自回归的方式生成每个字（即token），即将之前的生成结果作为下一步的输入，直至产生终止符。这种方式确保了模型在推理中的每一步都能充分利用上下文信息。为了提高推理效率，大模型推理引入了KV Cache机制，即缓存每一层已生成的Key和Value，这样在每一步计

算时不需要重复计算已生成的部分，从而大幅提升推理速度。如图3所示，第1步，“大”字作为大模型的输入并计算得到“模”字，计算得到的KV值被保存在显存中；第2步，前一步输出得到的“模”字与KV Cache作为输入并计得到“型”字。KV Cache的大小会随着序列生成过程逐步增加，但每个请求的序列长度难以预测，因此最终生成的KV Cache数据量难以确定。

大模型推理特有的KV Cache为存储系统带来了以下挑战。

（1）存储碎片化严重

首先，推理系统需为每个请求预留其KV Cache所需的显存空间，这会导致产生显存内部碎片。由于每一个请求的KV Cache数据量不可预测，系统需要预留最长输出序列对应的KV Cache所需的显存空间，以保证推理过程中的显存需求，但大部分预留显存在推理过程中不会被使用，这导致GPU显存内部碎片的产生。如图4所示，分配给请求1的显存空间仅存储了“大模型系”4个字对应的KV Cache，其他剩余的显存空间均被浪费。其次，剩余显存空间小于所需预留的显存大小，导致产生GPU内的外部碎片。例如，GPU1仅剩余4个字对应的KV Cache所需的显存空间，而新请求需要预留12个字对应的KV Cache所需的显存空间，这导致GPU1中剩余显存不能被使用。最后，GPU间也存在外部碎片。例如，虽然图4中的剩余显存容量总和可以满足新请求的显存预留需求，但这些剩余显存容量分布在多张GPU中，无法分配给单个新请求。

（2）KV Cache语义稀疏降低存储效率

大模型使用注意力机制计算KV Cache间的相关性分数，以建模句子中单

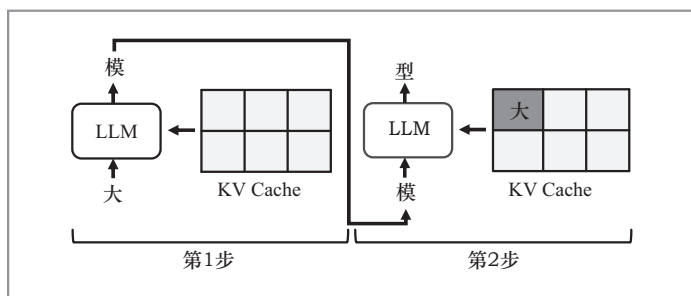


图3 大模型推理流程

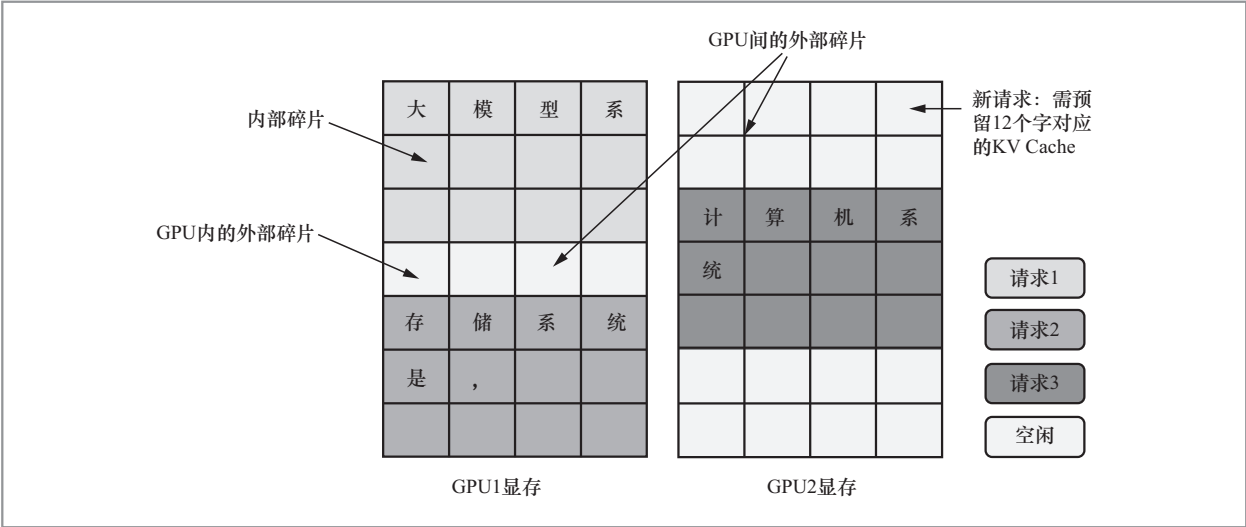


图4 推理中各类显存碎片

词之间的关系。然而，语言中存在大量无实义或信息重复的单词，它们与其他词的相关性较低，对计算的影响小。例如，“大模型的系统中”的“的”字在相关性分数表中与其他字的相关性低。根据研究人员测算，在主流模型中，KV Cache的稀疏度可达90%^[6]。在LLaMA-13B中，多达95%的KV Cache重要性较低，这些KV Cache可以被丢弃而不影响模型的精度。存储这些语义稀疏的KV Cache占用了大量的存储容量，且未对推理精度带来收益。

1.2 大模型数据管理技术

1.2.1 训练阶段的数据管理技术

本节将介绍现有工作如何解决大模型训练阶段临时数据存储效率低的问题。

(1) 前向传播激活量

现有研究基于“以算强存”的思想，采用前向传播激活量检查点与重算的方案，以缓解前向传播激活量长期占用显存的问题。具体而言，在前向传播过程中，系统仅保留少量前向传播激活量作为检查点，

其余激活量被释放。当反向传播需要使用这些已释放的前向传播激活量时，系统使用激活量检查点重新执行前向传播，恢复先前被丢弃的激活量。

在选择需要保留和丢弃的前向传播激活量时，Chen等^[7]提出了3种检查点选择算法：一是优先选择批归一化和池化等计算开销较低的操作，以避免大量额外计算成本；二是采用贪心策略将超出内存限制的激活量标记为检查点；三是使用递归手段逐步扩展子模型范围，寻找最优检查点方案。此外，伯克利大学提出的Checkmate^[8]将模型训练流程形式化为数学模型，并通过线性规划求解最优的激活量检查点配置方案。

(2) 反向传播梯度

针对反向传播中梯度长期占用显存的问题，现有研究将梯度数据暂存至DRAM或者SSD中。如微软提出的ZeRO-Offload系统^[9]将梯度数据卸载至DRAM。ZeRO-Offload的研究团队观察到：若在参数更新阶段将梯度数据重新传输至GPU显存，由于此阶段计算密度较低，数据迁

移带来的额外通信开销难以被计算掩盖。因此，ZeRO-Offload将计算密度较低的参数更新计算协同卸载至CPU。此外，为减少更新后参数上传至显存的开销，ZeRO-Offload采用了混合精度方案。具体来说，其采用FP32进行参数更新以保证模型精度，在数据传输时使用FP16以降低数据传输的成本。最后，ZeRO-Offload并行执行数据传输和计算，利用计算隐藏通信开销。

1.2.2 推理阶段的数据管理技术

(1) 减少KV Cache碎片

为减少单GPU的内部碎片与外部碎片，vLLM^[10]采用了块粒度的显存分配方案。具体来说，vLLM将用于存放KV Cache的物理显存划分为定长的块。在推理过程中，vLLM首先为每一个请求分配独立的虚拟显存空间。随着请求输出序列的增长，vLLM按需为每块虚拟显存分配对应的物理显存，并将虚拟显存块与物理显存块的映射记录在映射表中。如图5所示，请求1的虚拟显存中的块1映射到GPU物理显存中的块2。而请求1和请求2的虚拟显存中的块3均未被使用，因此暂时未为其分配实际的GPU物理显存块，这减少了GPU显存的内部碎

片。此外，vLLM利用映射表减少请求间的冗余数据存储。具体来说，若不同请求中包含相同序列，vLLM会将这些相同序列对应的KV Cache虚拟显存块映射至相同的物理显存。如图5所示，请求1和请求2的虚拟显存中块1均存储了“大模型系”对应的KV Cache，因此vLLM将请求1和请求2的虚拟显存块1映射到相同的物理显存块2，这进一步降低了GPU的显存容量开销。

针对GPU间的外部显存碎片，Llumnix^[11]提出了GPU间动态请求迁移方案。其核心思想是：将KV Cache在不同GPU间进行迁移，以合并不同GPU内的显存碎片，从而满足新请求的显存需求。如图6所示，新到达的请求需要预留12个字对应的KV Cache大小的显存空间。虽然GPU1和GPU2中的剩余显存空间总和可以满足新请求的存储容量需求，但这些剩余的显存空间分散在不同的GPU上，无法直接分配给新的请求。因此，Llumnix将GPU1中的某一请求对应的KV Cache迁移至GPU2中，并将2个GPU的剩余显存空间集中至GPU1中，以满足新请求KV Cache的存储容量需求。为隐藏请求在不同GPU间迁移带来的通信

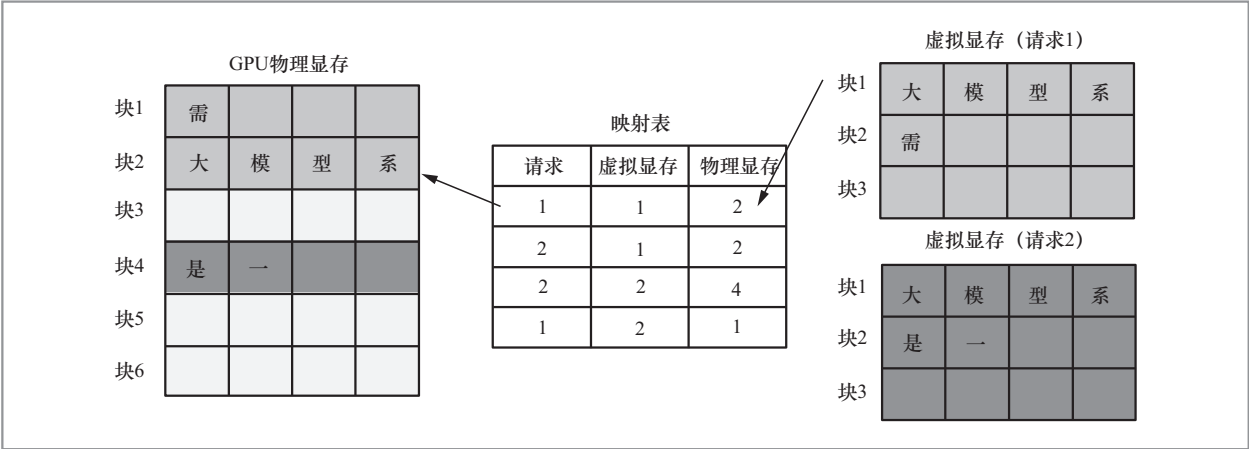


图5 块粒度按需分配显存

开销，Llumnix并行进行计算和KV Cache迁移，以使用计算隐藏通信开销。

(2) 提高存储数据的语义密度

现有研究基于语义信息对KV Cache进行缩减，从而提高存储的语义密度。这些方法主要分为两类。

一类使用剪枝技术来缩减KV Cache。剪枝方法分为基于窗口的剪枝^[12]、基于重要性的剪枝^[6]以及基于金字塔结构的剪枝^[13]。基于窗口的剪枝仅保留序列的开头与窗口长度内的KV Cache；基于重要性的剪枝根据重要程度删除对结果无贡献的KV Cache。在这两种方案中，模型每一层删减的KV Cache的位置均相同。而基于金字塔结构的剪枝则放宽了这一限制，该方法在每一层保留的KV Cache的数量和位置均不同。例如，它会保留底层所有的KV Cache，以及顶层较少的关键KV Cache。

另一类通过修改大模型推理流程减少KV Cache的存储需求。其中，清华大学提出了基于隐藏状态的KV Cache恢复方案^[14]。如图7所示，其利用输入压缩得到中间状态代替KV Cache；在计算时，通过重计算的方式恢复Key与Value，从而进行标准的注意力计算。该方案在仅增加15%计算开销的前提下，可将KV Cache体积压缩50%。

2 大模型时代下的存储扩容

除显存外，GPU服务器里还有DRAM、SSD等其他的存储介质，这些存储介质的容量更大且价格更低^[15-16]，并且其扩展更加灵活，可以与GPU计算资源扩展解耦。在大模型训练与推理过程中使用DRAM或SSD可以支持更大规模的模型。本节将分析在大模型训练与推理中使用DRAM或SSD带来

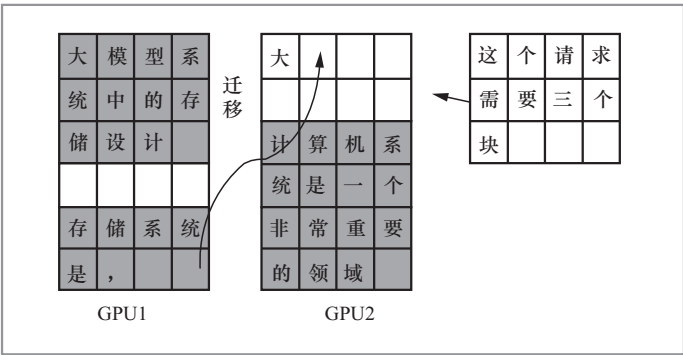


图6 GPU间动态请求迁移

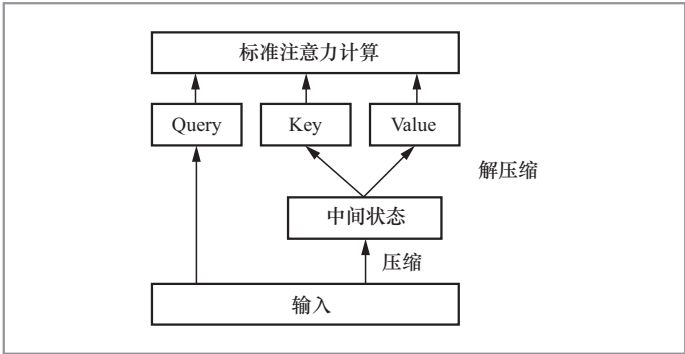


图7 基于隐藏状态的KV Cache恢复方案

的挑战，并介绍现有研究的解决方案。

2.1 大模型时代下的存储扩容挑战

受限于链路带宽或存储介质的读写带宽，GPU读写DRAM与SSD的带宽远低于GPU读写显存的带宽。如表1所示，GPU读写本地显存的带宽为TB/s级别，而GPU读写DRAM与SSD的带宽为10 GB/s级别。

然而，大模型训练与推理对数据的读写带宽要求高，直接使用异构存储介质会影响GPU的计算。以GPT-3为例，在训练阶段，单步训练需要访问约1 TB的数据（包括模型参数、激活量、优化器参数以及梯度等）。通过PCIe（32 GB/s）在GPU显存与DRAM之间传输这些数据需要约

表1 A100 GPU访问不同存储介质的读写带宽

访问介质	链路	读带宽/(GB·s ⁻¹)	写带宽/(GB·s ⁻¹)
本地显存	GPU 内部总线	1 638	1 638
其他GPU 显存	NVLink	300	300
	PCIe 4.0 × 16	32	32
本地 DRAM	PCIe 4.0 × 16	32	32
本地 SSD	PCIe 4.0 × 4	6.9	4.1
远程 DRAM	InfiniBand	25	25
	Ethernet	1.25	1.25

32 s，而 GPT-3 单步训练的计算时间仅约 10 s。在推理阶段，当 GPT-3 的推理批大小为 32 且生成序列长度为 2 000 个字时，推理过程会产生约 200 GB 的 KV Cache。通过 PCIe 在 GPU 显存与 DRAM 之间传输所有模型参数和 KV Cache 需要约 17 s，而用户的单字推理延迟需求通常仅为 100 ms 级别。因此，直接使用异构存储介质会导致数据传输开销远高于计算开销，使数据通信成为瓶颈，进而导致 GPU 计算资源无法被充分利用。

2.2 大模型时代下的存储扩容技术

大模型训练通常采用批处理的方式以充分利用 GPU 计算资源，因此其对数据读写吞吐需求高；推理阶段则更注重低延迟，以提升用户体验。因而，针对大模型的存储扩容工作被分为训练阶段的存储扩容和推理阶段的存储扩容。

2.2.1 训练阶段的存储扩容

现有系统会利用 GPU 服务器中的聚合带宽与高速链路对显存进行扩容，以满足大模型训练阶段对吞吐的要求。GPU 集群中聚合 PCIe 带宽与服务器数量成正比，例如

64 台 DGX-2 的聚合 PCIe 带宽为 4 TB/s。此外，GPU 集群中还存在一些高速链路。例如，GPU 间高速互联的通信链路 NVLink 可实现单向 300 GB/s 的 GPU 间通信；通过 InfiniBand 互联的服务器间的通信带宽可达 25 GB/s。为充分利用上述通信资源，微软提出的高性能训练系统 ZeRO-Infinity^[17]将模型数据条带化存储至多个 DRAM 或者 SSD 上，以利用聚合 PCIe 带宽将数据快速上传至 GPU 显存，利用高速链路聚合每张 GPU 上由 PCIe 上传的分块数据。

在 ZeRO-Infinity 中，单张 GPU 需要向所有 GPU 广播模型参数与梯度数据，导致 GPU 间数据通信量大。然而，在不支持 NVLink 的 GPU（如消费级 GPU）上，GPU 的卡间通信以及 GPU 与异构存储介质间的通信共用链路。频繁的 GPU 卡间通信会加剧异构存储介质间的数据传输开销。为解决上述问题，清华大学提出了面向消费级 GPU 服务器的大模型训练系统 Mobius^[18]。Mobius 采用了基于异构存储介质的流水线并行训练策略，显著减少了训练过程中 GPU 间的数据通信量。Mobius 将数据传输与模型计算并行化，以隐藏数据传输带来的开销。此外，Mobius 设计了拓扑感知的通信调度策略，将流水线中相邻的通信散布至处于不同 CPU Root Complex 上的 GPU 中，以减少通信竞争。Mobius 会优先传输先被使用的数据，以减少通信对计算的阻塞。

2.2.2 推理阶段的存储扩容

在面向推理的显存扩容研究中，有一类研究聚焦于将模型参数卸载至异构存储介质。研究人员发现^[19]，在推理过程中，每层模型参数计算得到的向量中大量位置的值为 0。例如 OPT 模型中，每层输出的

向量中超过90%的位置的数值为0。因此，部分研究使用预测器^[20]，基于每层的输入向量预测每层输出向量的非零位置，然后仅将非零位置对应的模型参数上传至显存用于计算。这显著降低了数据在异构存储介质之间的传输量。

还有一类研究将KV Cache卸载至异构存储介质。受限于PCIe的带宽，卸载KV Cache至DRAM或者SSD会引入相较于计算近50倍的传输开销，这会浪费GPU的计算资源并且增加推理延迟。为降低卸载KV Cache带来的通信开销，InfiniGen^[21]基于KV Cache的语义信息，动态计算每轮推理中每个token对应的KV Cache的重要性，并仅上传关键的token对应的KV Cache至GPU显存用于推理。此外，部分研究通过存算协同卸载的方式降低通信开销。清华大学的研究人员发现^[22]：从硬件角度来说，相较于GPU，CPU单位算力对应的存储容量更高；从算法角度来说，相较于非KV Cache相关的算子，KV Cache相关算子单位计算所需的存储容量更高。因此，CPU的存算资源比和KV Cache相关算子的存算需求比更吻合。基于此，FastDecode^[22]使用CPU DRAM存储KV Cache，并将KV Cache相关算子的计算协同卸载至CPU，仅上传KV Cache相关算子的计算结果至GPU。相较于传输KV Cache至GPU显存，FastDecode可以显著降低通信量。

3 大模型时代的数据容错

3.1 大模型时代的数据容错需求与挑战

大模型训练需要耗费大量时间和海量的

计算资源，显著提高了训练过程中出现软硬件故障的风险。例如，OPT-175B模型训练使用了992张A100 80 GB GPU，训练过程中共发生了105次故障，其中最长的一段连续无故障训练时间仅为2.8天。然而，由于GPU显存属于易失存储介质，故障会造成训练数据丢失。此外，大模型训练通常采取分布式训练模式，故障会扩散到整个集群中。GPU训练中有两类不同的通信模式，即点到点通信和集合通信。如图8所示，在点到点的通信模式中，失效GPU的前继GPU的计算结果无法继续向下传递，而后继GPU无法得到新的输入用于计算；在集合通信模式中，同步阶段无法得到失效GPU的计算结果，从而阻塞后续的计算。

大模型训练过程中的数据容错需求高，但是直接使用以CPU为中心的容错技术会带来高昂的开销。以CPU为中心的计算应用通常采用检查点技术对数据进行容错。以高性能计算中的检查点技术为例，系统会定期暂停计算任务，并将内存中的数据、进程状态和计算进度等保存至持久性存储介质中；当系统发生故障时，计算任务可以从最近的检查点开始重新执行，从而节省计算资源和时间。随着大模型规模的不断扩大，训练中数据量和计算量急剧增加，大模型的训练严重依赖计算高度并行的GPU。然而，GPU读写持久化存储介质的带宽远低于其算

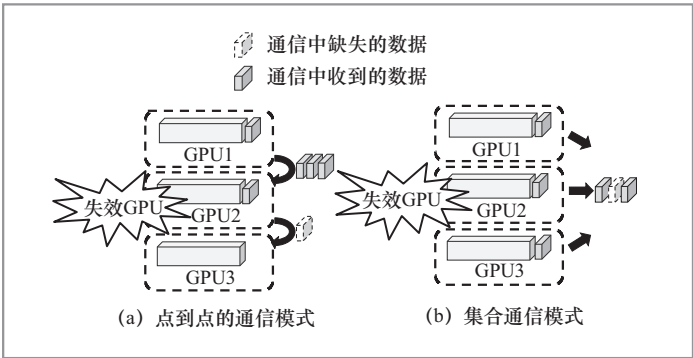


图8 两类故障模式

力带宽,这暴露了传统检查点技术的局限性。具体而言,由于GPU与持久化存储介质间的数据传输带宽较低,传统检查点技术在保存或恢复检查点时,数据I/O会长时间阻塞计算任务,导致计算资源被严重浪费。根据UIUC的测算^[23],若在大模型训练中使用传统检查点技术,数据传输时长会占据训练总时长的70%。

3.2 大模型时代下的数据容错技术

为降低大模型训练过程中的容错开销,一类方法基于大模型训练特征重新设计检查点技术,以减少检查点的写入开销;另一类方法通过多张GPU上重复大模型训练中的计算任务,以实现模型训练结果的冗余,从而避免检查点带来额外的I/O开销。

(1) 检查点技术

针对大模型训练检查点写入的优化,有研究基于大模型训练流程,设计了异步持久化检查点技术,减少容错过程中I/O对计算的阻塞。多伦多大学提出的CheckFreq系统^[24],将检查点写入过程分为快照和持久化两个阶段,以避免因数据持久化开销过高导致的计算阻塞。具体而言,在快照阶段系统将更新后的参数副本保存在显存或者内存中;在持久化阶段,系统再将快照的模型参数写入持久化介质中。由于快照后的模型数据与训练过程中使用的模型数据是相互独立的,持久化操作可以在后台进行,不影响后续的计算任务。

此外,还有研究利用大模型并行训练特征,加速检查点I/O吞吐。微软DeepSpeed团队提出的FastPersist系统^[25]利用数据并行训练模式下多GPU共享相同模型数据的特性,将同一检查点数据的不同分块并行写入多个GPU中,利用聚合PCIe带宽加速检查点写入过程。

(2) 冗余计算

部分研究基于大模型并行训练的特征,在无须持久化写入检查点的情况下实现容错功能。加州大学洛杉矶分校提出的Bamboo系统^[23]利用冗余计算机制代替传统的检查点持久化,以实现容错功能。该机制利用大模型流水线并行训练策略中产生的计算气泡,在相邻节点上对模型数据进行冗余计算;当某个节点发生故障时,其相邻节点可以代为执行故障节点的前向计算和反向计算。密歇根大学提出的Oobleck系统^[26]利用大模型数据并行训练策略中的冗余模型数据代替持久化的检查点。具体地,当某个数据并行组内的GPU出现故障时,该系统直接从其他并行组的GPU中拉取模型参数,从而恢复训练任务。

4 结束语

本文详细阐述了大模型时代下存储系统面临的挑战,并介绍了现有大模型存储技术在数据管理、容量扩展以及数据容错方面的进展。随着大模型的持续发展,大模型时代的存储系统可在以下方面进行深入研究。

(1) 绿色存储

大模型的全生命周期需要大量的存储设备,以满足海量数据的存储需求。在模型训练阶段,需要高读写带宽的显存(如HBM)来存储训练数据,以匹配GPU的高算力;在模型推理阶段,需要大量显存来存储模型参数和上下文信息(即KV Cache),以满足推理的低时延需求。然而,大规模存储设备的使用会产生大量的碳排放。为实现大模型全周期的绿色云存储,一方面,需要考虑如何在大模型全生

命周期中进行存算协同设计,以减少存储需求;另一方面,还需探讨如何在满足性能要求的前提下,选择能耗更低的存储介质。

(2) 云端协同存储

在模型训练阶段,云端协同存储可以为用户提供个性化的大模型微调服务,同时充分利用端侧算力降低服务提供商的训练成本。在模型推理阶段,云端协同存储可以为用户提供离线推理服务,以保护用户隐私。然而,云端协同存储面临一些新的问题,比如如何分配云端两侧的存算任务、如何降低云端两侧的数据传输开销等。

(3) 系统与算法的协同设计

在大模型时代,算法领域需要面向硬件和系统特性重新设计大模型的训练与推理算法,使大模型训练与推理过程中的存算需求吻合硬件和系统特性,从而充分发挥硬件和系统的性能。此外,硬件系统领域也需要针对大模型的算法特性重新设计领域专用的硬件和系统,以提高大模型训练与推理性能。

参考文献:

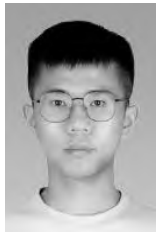
- [1] KAPLAN J, MCCANDLISH S, HENIGHAN T, et al. Scaling laws for neural language models[EB]. arXiv preprint, 2020, arXiv: 2001.08361.
- [2] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[C]//Proceedings of the 31st International Conference on Neural Information Processing Systems. Cambridge: MIT Press, 2017: 6000–6010.
- [3] 冯杨洋, 汪庆, 谢旻晖, 等. 从 BERT 到 ChatGPT: 大模型训练中的存储系统挑战与技术发展[J]. 计算机研究与发展, 2024, 61(4): 809–823.
- FENG Y Y, WANG Q, XIE M H, et al.

From BERT to ChatGPT: challenges and technical development of storage systems for large model training[J]. Journal of Computer Research and Development, 2024, 61(4): 809–823.

- [4] JEON M, VENKATARAMAN S, PHANISHAYEE A, et al. Analysis of large-scale multi-tenant GPU clusters for DNN training workloads[C]//Proceedings of 2019 USENIX Annual Technical Conference. Berkeley: USENIX Association, 2019: 947–960.
- [5] XIE M H, REN K, LU Y Y, et al. Kraken: memory-efficient continual learning for large-scale real-time recommendations[C]//Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. Piscataway: IEEE, 2020: 1–17.
- [6] ZHANG Z Y, SHENG Y, ZHOU T Y, et al. H₂O: heavy-hitter oracle for efficient generative inference of large language models[EB]. arXiv preprint, 2023, arXiv: 2306.14048.
- [7] CHEN T Q, XU B, ZHANG C Y, et al. Training deep nets with sublinear memory cost[EB]. arXiv preprint, 2016, arXiv: 1604.06174.
- [8] JAIN P, JAIN A, NRUSIMHA A, et al. Checkmate: breaking the memory wall with optimal tensor rematerialization[EB]. arXiv preprint, 2019, arXiv: 1910.02653.
- [9] REN J, RAJBHANDARI S, AMINABADI R Y, et al. ZeRO-offload: democratizing billion-scale model training[EB]. arXiv preprint, 2021, arXiv: 2101.06840.
- [10] KWON W, LI Z H, ZHUANG S Y, et al. Efficient memory management for large language model serving with PagedAttention[EB]. arXiv preprint, 2023, arXiv: 2309.06180.
- [11] SUN B, HUANG Z M, ZHAO H Y, et al. Llumnix: dynamic scheduling for large language model serving[C]//Proceedings

- of 18th USENIX Symposium on Operating Systems Design and Implementation. Berkeley: USENIX Association, 2024: 173–191.
- [12] XIAO G X, TIAN Y D, CHEN B D, et al. Efficient streaming language models with attention sinks[EB]. arXiv preprint, 2020, arXiv: 2309.17453.
- [13] YANG D J, HAN X D, GAO Y, et al. PyramidInfer: pyramid KV cache compression for high-throughput LLM inference[EB]. arXiv preprint, 2024, arXiv: 2405.12532.
- [14] GAO S W, CHEN Y M, SHU J W. Fast state restoration in LLM serving with HCache[EB]. arXiv preprint, 2024, arXiv: 2410.05004.
- [15] XIE M H, LU Y Y, WANG Q, et al. PetPS: supporting huge embedding models with persistent memory[J]. Proceedings of the VLDB Endowment, 2023, 16(5): 1013–1022.
- [16] XIE M H, LU Y Y, LIN J Z, et al. Fleche: an efficient GPU embedding cache for personalized recommendations [C]//Proceedings of the Seventeenth European Conference on Computer Systems. New York: ACM, 2022: 402–416.
- [17] RAJBHANDARI S, RUWASE O, RASLEY J, et al. ZeRO-Infinity: breaking the GPU memory wall for extreme scale deep learning[EB]. arXiv preprint, 2021, arXiv: 2104.07857.
- [18] FENG Y Y, XIE M H, TIAN Z J, et al. Mobius: fine tuning large-scale models on commodity GPU servers[C]//Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. New York: ACM, 2023: 489–501.
- [19] LIU Z C, WANG J, DAO T, et al. Deja Vu: contextual sparsity for efficient LLMs at inference time[C]//Proceedings of the 40th International Conference on Machine Learning. New York: ACM, 2023: 22137–22176.
- [20] ALIZADEH K, MIRZADEH I, BELENKO D, et al. LLM in a flash: efficient large language model inference with limited memory[EB]. arXiv preprint, 2023, arXiv: 2312.11514.
- [21] LEE W, LEE J, SEO J, et al. InfiniGen: efficient generative inference of large language models with dynamic KV cache management[C]//Proceedings of 18th USENIX Symposium on Operating Systems Design and Implementation. Berkeley: USENIX Association, 2024: 155–172.
- [22] HE J A, ZHAI J D. FastDecode: high-throughput GPU-efficient LLM serving using heterogeneous pipelines[EB]. arXiv preprint, 2024, arXiv: 2403.11421.
- [23] THORPE J, ZHAO P, EYOLFSON J, et al. Bamboo: making preemptible instances resilient for affordable training of large DNNs[C]//Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation. Berkeley: USENIX Association, 2023: 497–513.
- [24] MOHAN J, PHANISHAYEE A, CHIDAMBARAM V. CheckFreq: frequent, fine-grained DNN checkpointing[C]//Proceedings of the 19th USENIX Conference on File and Storage Technologies. Berkeley: USENIX Association, 2021: 203–216.
- [25] WANG G H, RUWASE O, XIE B, et al. FastPersist: accelerating model checkpointing in deep learning[EB]. arXiv preprint, 2024, arXiv: 2406.13768.
- [26] JANG I, YANG Z N, ZHANG Z, et al. Ooblock: resilient distributed training of large models using pipeline templates [C]//Proceedings of the 29th Symposium on Operating Systems Principles. New York: ACM, 2023: 382–395.

作者简介



冯杨洋（1998-），男，清华大学计算机科学与技术系博士生，主要研究方向为存储系统和机器学习系统。



汪庆（1997-），男，清华大学计算机科学与技术系博士后，主要研究方向为存储系统和内存系统。



舒继武（1968-），男，博士，清华大学计算机科学与技术系教授、博士生导师，中国计算机学会（CCF）会士，主要研究方向为非易失内存存储系统、存储可靠性和分布式计算。

收稿日期：2024-09-26

通信作者：舒继武，shujw@tsinghua.edu.cn

基金项目：国家自然科学基金联合基金项目(No.U22B2023)

Foundation Item: The Joint Funds of the National Natural Science Foundation of China (No.U22B2023)